Gruplab - Directivas del equipo (Actualizado al estado actual del proyecto)

Documento de trabajo para los 5 integrantes. Actualizado con el avance real del repositorio (rama develop / luciano-aulas). Define que hace cada uno, que archivos toca, que tablas crea, que endpoints expone, que componentes construye y como se conecta con los demas.

Stack: Node.js + Express + SQL Server (mssql) en backend, Angular 21 Standalone + Bootstrap 5.3 en frontend, JavaScript (CommonJS) y TypeScript.

---

0. Indice

1. Estado actual del repositorio
2. Reparto de modulos (5 integrantes Fullstack con tablas propias)
3. Reglas de Git
4. Archivos compartidos
5. Convenciones de backend (con plantilla real del proyecto)
6. Convenciones de frontend (Angular 21 + inject + Signals + Bootstrap)
7. Base de datos completa (schema.sql)
8. Contratos de integracion entre modulos
9. Modulo 1 - Luciano - Aulas y Home (COMPLETADO)
10. Modulo 2 - Leandro - Grupos (Backend listo / Falta Frontend)
11. Modulo 3 - Gonzalo - Miembros (Backend listo / Falta Frontend)
12. Modulo 4 - Abel - Tablero Kanban y Columnas Dinamicas (BoardColumns)
13. Modulo 5 - Leonardo - ABM de Tareas y Comentarios (Tasks y Comments)
14. Como trabajar sin esperar al companero (datos de prueba)
15. Checklist de entrega por integrante

---

1. Estado actual del repositorio

```
gruplab-fe-be/
  README.md
  backend/
    .env                          (no se sube, cada uno tiene el suyo)
    package.json                  (express, mssql, cors, dotenv, nodemon)
    peticionesTest.http           (pruebas unificadas de todos los modulos)
    src/
      index.js                    (servidor Express, middlewares y registro de rutas)
      database/
        config.js                 (dotenv y dbConfig)
        connection.js             (getConnection y sql con manejo de errores)
        queries.js                (queries unificadas de todos los modulos)
        schema.sql                (tablas SQL del proyecto)
      routes/
        classroom.routes.js       (LISTO - Luciano)
        group.routes.js           (LISTO - Leandro)
        member.routes.js          (LISTO - Gonzalo)
        board.routes.js           (Pendiente - Abel: Columnas y Tablero)
        task.routes.js            (Pendiente - Leonardo: Tareas)
        comment.routes.js         (Pendiente - Leonardo: Comentarios)
      controllers/
        classroom.controller.js   (LISTO - Luciano)
        group.controller.js       (LISTO - Leandro)
        member.controller.js      (LISTO - Gonzalo)
```

```
        board.controller.js      (Pendiente - Abel)
        task.controller.js       (Pendiente - Leonardo)
        comment.controller.js    (Pendiente - Leonardo)
      services/
        classroom.service.js     (LISTO - Luciano)
        group.service.js         (LISTO - Leandro)
        member.service.js        (LISTO - Gonzalo)
        board.service.js         (Pendiente - Abel)
        task.service.js          (Pendiente - Leonardo)
        comment.service.js       (Pendiente - Leonardo)
      repositories/
        classroom.repository.js  (LISTO - Luciano)
        group.repository.js      (LISTO - Leandro)
        member.repository.js     (LISTO - Gonzalo)
        board.repository.js      (Pendiente - Abel)
        task.repository.js       (Pendiente - Leonardo)
        comment.repository.js    (Pendiente - Leonardo)
  frontend/
    src/
      index.html                 (incluye Bootstrap 5.3 CSS via CDN)
      main.ts
      app/
app.ts / app.html        (Header con RouterLink a /classrooms, router-outlet y Footer)
        app.config.ts            (provideRouter y provideHttpClient)
        app.routes.ts            (Rutas con carga perezosa / lazy loading)
        features/
          classrooms/            (MODULO 1 COMPLETO - Luciano)
models/classroom.ts  (Interfaces: Classroom, CrearAulaResponse, UsuarioSession, AuLab)
services/classroom.ts(ClassroomService con Observable<T> y manejo de localStorage)
            home/                (Home + listado multirrol "Mis AuLabs")
            profesor/            (Formulario de creacion de aula)
            alumno/              (Formulario de ingreso por codigo)
panel-aula/          (Panel del aula con permisos de profesor/alumno y espacio para
Grupos y Miembros)
          groups/                (Pendiente Frontend - Leandro)
          members/               (Pendiente Frontend - Gonzalo)
          tasks/                 (Compartido sin pisarse entre Abel y Leonardo)
            models/board-column.ts (Pendiente - Abel: Interfaz BoardColumn)
            models/task.ts       (Pendiente - Leonardo/Abel: Interfaz Task)
            services/board.ts    (Pendiente - Abel: BoardService)
            services/task.ts     (Pendiente - Leonardo: TaskService)
kanban/              (Pendiente - Abel: Tablero con 4 columnas base + agregar columna)
task-card/           (Pendiente - Abel: Tarjeta de tarea y movimiento entre columnas)
            task-form/           (Pendiente - Leonardo: Crear tarea)
            task-detail/         (Pendiente - Leonardo: Ver/Editar/Borrar tarea)
          comments/              (Pendiente Frontend - Leonardo)
            models/comment.ts
            services/comment.ts
            comments/
```

Puertos:
- Backend: http://localhost:3000 (npm run dev dentro de backend)
- Frontend: http://localhost:4200 (npm start dentro de frontend)

---

2. Reparto de modulos (5 integrantes Fullstack con tablas propias)

Cada integrante es duenio de sus propias tablas SQL, su propio Backend (Repository + Service + Controller + Routes) y su propio Frontend:

Integrante	Modulo	Tablas SQL	Archivos Backend	Carpeta / Componentes Frontend	Estado actual

Luciano	Aulas, Home y Panel del Aula	Classrooms	classroom.*	features/classrooms (Home, Profesor, Alumno, PanelAula)	100% Completado
Leandro	Grupos de trabajo	Groups	group.*	features/groups (GroupService, GroupList)	Backend listo / Falta Front
Gonzalo	Miembros del aula	Members	member.*	features/members (MemberService, MemberList)	Backend listo / Falta Front
Abel	Tablero Kanban y Columnas Dinamicas (4 columnas iniciales + crear/editar/borrar columna y mover tarea)	BoardColumns	board.* (board.repository, board.service, board.controller, board.routes)	features/tasks (BoardService, Kanban, TaskCard)	Pendiente
Leonardo	ABM de Tareas y Comentarios	Tasks, Comments	task.* y comment.*	features/tasks (TaskService, TaskForm, TaskDetail) y features/comments (CommentService, Comments)	Pendiente

---

3. Reglas de Git

Ramas:
- main: version estable. Solo se mergea desde develop cuando todo funciona.
- develop: integracion de todos los modulos.
- Ramas personales: luciano-aulas, leandro-grupos, gonzalo-miembros, abel-tablero, leo-comments.

Flujo diario:
1. Pararse en la rama propia: git checkout <tu-rama>
2. Traer lo ultimo de develop antes de empezar: git pull origin develop
3. Trabajar y hacer commits descriptivos: git commit -m "tablero: crud de columnas y movimiento de tareas"
4. Subir la rama propia: git push origin <tu-rama>
5. Abrir Pull Request hacia develop.

---

4. Archivos compartidos (donde hay riesgo de pisarse)

Regla: solo se AGREGAN lineas al final del bloque correspondiente, nunca se borran lineas de otro.

Archivo	Que agrega cada uno
backend/src/index.js	require y app.use de sus rutas (board.routes, task.routes, comment.routes)
backend/src/database/schema.sql	CREATE TABLE BoardColumns (Abel), CREATE TABLE Tasks y CREATE TABLE Comments (Leonardo)
backend/src/database/queries.js	Sus queries al final del module.exports
backend/peticionesTest.http	Su bloque de pruebas al final del archivo
frontend/src/app/app.routes.ts	La ruta con loadComponent para /groups/:groupId/board (antes de '')

---

5. Convenciones de backend (Patron real del proyecto)

5.1 Capas y manejo de errores

| Capa | Archivo | Responsabilidad | Manejo de Errores |

|---|---|---|---|
Queries	database/queries.js	Sentencias SQL con parametros @ y constantes queryBaseSelect	-
Repository	repositories/x.repository.js	Ejecutar SQL con mssql y devolver resultado.recordset	Envuelve en try/catch y lanza new Error('Mensaje limpio') para no filtrar errores de SQL Server
Service	services/x.service.js	Desestructurar const [item] = ..., validar datos y reglas de negocio	Sin try/catch. Solo valida con if y hace throw new Error('...')
Controller	controllers/x.controller.js	Leer req.params, req.query y req.body || {}, llamar al service y responder	Unico try/catch: responde 200 o 400 con { error: error.message }
Routes	routes/x.routes.js	Asociar metodo HTTP + URL con el controller	-

5.2 Plantilla real del proyecto (copiar este estilo exacto)

En repositories/board.repository.js:

```
const { sql, getConnection } = require('../database/connection');
const queries = require('../database/queries');

exports.obtenerColumnasPorGrupo = async (groupId) => {
    const pool = await getConnection();
    try {
        const resultado = await pool.request()
            .input('group_id', sql.Int, groupId)
            .query(queries.obtenerColumnasPorGrupo);
        return resultado.recordset;
    } catch (error) {
        throw new Error('Error al obtener las columnas del tablero');
    }
};
```

En services/board.service.js:

```
const boardRepository = require('../repositories/board.repository');

exports.crearColumna = async (groupId, name, color) => {
    if (!groupId || !name || !name.trim()) {
        throw new Error('El nombre de la columna es obligatorio');
    }
    const columnasActuales = await boardRepository.obtenerColumnasPorGrupo(groupId);
    const nuevaPosicion = columnasActuales.length + 1;
    const colorFinal = color || 'secondary';

    const [columnaCreada] = await boardRepository.crearColumna(
        groupId,
        name.trim(),
        nuevaPosicion,
        colorFinal
    );
    return {
        message: 'Columna creada exitosamente',
        id: columnaCreada.id,
        position: nuevaPosicion
    };
};
```

En controllers/board.controller.js:

```
const boardService = require('../services/board.service');

exports.crearColumna = async (req, res) => {
    try {
        const { groupId } = req.params;
        const { name, color } = req.body || {};
        const resultado = await boardService.crearColumna(groupId, name, color);
        res.status(200).json(resultado);
```

```
    } catch (error) {
        res.status(400).json({ error: error.message });
    }
};
```

---

6. Convenciones de frontend (Angular 21 Standalone + Bootstrap 5.3)

6.1 Reglas actualizadas del proyecto

1. Componentes Standalone: standalone: true con sus propios imports: [...].
2. Inyeccion de dependencias con inject(): No usar constructores para inyectar. Usar:

```
    private boardService = inject(BoardService);
    private route = inject(ActivatedRoute);
    private router = inject(Router);
```

3. Interfaces TypeScript (models/): Crear interfaces para tipar los modelos y respuestas. No dejar todo en any.
4. Servicios con Observable<T>: Tipar el retorno de cada metodo HTTP (Observable<BoardColumn[]>, Observable<Task[]>, etc.).
5. Estado con Signals: columnas = signal<BoardColumn[]>([]);, tareas = signal<Task[]>([]);.
6. Ciclo de vida OnInit: Hacer las llamadas HTTP iniciales dentro de ngOnInit(): void.
7. Lectura de parametros de ruta con .subscribe():

```
    ngOnInit(): void {
      this.route.params.subscribe(params => {
        const groupId = Number(params['groupId']);
        this.cargarTablero(groupId);
      });
    }
```

8. Carga Perezosa (Lazy Loading) en app.routes.ts:

```
    {
      path: 'groups/:groupId/board',
      loadComponent: () => import('./features/tasks/kanban/kanban').then(m => m.Kanban)
    }
```

9. Estilos con Bootstrap 5.3: Ya esta cargado en index.html. Usar clases nativas de Bootstrap (card, shadow-sm, btn, badge, form-control, row, col).

---

7. Base de datos completa (`schema.sql`)

Las tablas 1 (Classrooms), 2 (Groups) y 3 (Members) ya estan creadas en schema.sql. Se agregan al final la tabla 4 (BoardColumns de Abel) y las tablas 5 y 6 (Tasks y Comments de Leonardo):

```
-- 1. Luciano (YA EXISTE)
CREATE TABLE Classrooms (
    id        INT           IDENTITY (1, 1) PRIMARY KEY,
    name      VARCHAR (100) NOT NULL,
    duenio    VARCHAR (100) NOT NULL,
    join_code VARCHAR (20)  NOT NULL UNIQUE
);

-- 2. Leandro (YA EXISTE)
CREATE TABLE Groups (
    id           INT           IDENTITY (1, 1) PRIMARY KEY,
classroom_id INT           NOT NULL FOREIGN KEY REFERENCES Classrooms (id) ON DELETE
CASCADE,
    group_number INT           NOT NULL,
    project_name VARCHAR (100) NOT NULL,
```

```
    UNIQUE (classroom_id, group_number)
);

-- 3. Gonzalo (YA EXISTE)
CREATE TABLE Members (
    id           INT           IDENTITY (1, 1) PRIMARY KEY,
    classroom_id INT           NOT NULL FOREIGN KEY REFERENCES Classrooms (id),
group_id     INT           NULL FOREIGN KEY REFERENCES Groups (id) ON DELETE SET NULL,
    nombre       VARCHAR (100) NOT NULL,
    email        VARCHAR (100) NULL,
    rol          VARCHAR (20)  DEFAULT 'alumno' NOT NULL,
    created_at   DATETIME      DEFAULT GETDATE()
);

-- 4. Abel (AGREGAR AHORA - Columnas del tablero de cada grupo)
CREATE TABLE BoardColumns (
    id         INT          IDENTITY (1, 1) PRIMARY KEY,
group_id   INT          NOT NULL FOREIGN KEY REFERENCES Groups (id) ON DELETE CASCADE,
name       VARCHAR (50) NOT NULL, -- 'Por hacer', 'En progreso', 'En revision',
'Terminado' o nuevas
    position   INT          NOT NULL, -- Orden de izquierda a derecha: 1, 2, 3, 4...
color      VARCHAR (20) DEFAULT 'secondary' NOT NULL -- 'secondary', 'primary',
'warning', 'success', 'info'
);

-- 5. Leonardo (AGREGAR AHORA - Tareas dentro de una columna del tablero)
CREATE TABLE Tasks (
    id          INT            IDENTITY (1, 1) PRIMARY KEY,
    group_id    INT            NOT NULL,
column_id   INT            NOT NULL FOREIGN KEY REFERENCES BoardColumns (id) ON DELETE
CASCADE,
    assigned_to INT            NULL FOREIGN KEY REFERENCES Members (id),
    title       VARCHAR (150)  NOT NULL,
    description VARCHAR (1000) NULL,
    created_at  DATETIME       DEFAULT GETDATE()
);

-- 6. Leonardo (AGREGAR AHORA - Comentarios de cada tarea)
CREATE TABLE Comments (
    id          INT            IDENTITY (1, 1) PRIMARY KEY,
task_id     INT            NOT NULL FOREIGN KEY REFERENCES Tasks (id) ON DELETE CASCADE,
    author_name VARCHAR (100)  NOT NULL,
    content     VARCHAR (1000) NOT NULL,
    created_at  DATETIME       DEFAULT GETDATE()
);
```

> Nota sobre cascadas en SQL Server (sin error 1785):
> La cadena de borrado en cascada es lineal: Classrooms -> Groups -> BoardColumns -> Tasks -> Comments.
> Tasks.assigned_to no lleva ON DELETE SET NULL porque SQL Server no permite caminos multiples; ya esta resuelto en queries.js con liberarTareasDelMiembro.

---

8. Contratos de integracion entre modulos

8.1 Sesion activa y Mis AuLabs (`localStorage`)

Ya implementado por Luciano en ClassroomService:
- Clave de sesion activa: gruplab_usuario

```
  {
    "nombre": "Lautaro",
    "rol": "profesor",
    "classroomId": 1,
```

```
  "classroomName": "Programacion Web",
  "memberId": 1
}
```

- Clave de historial multirrol: gruplab_mis_aulabs (permite actuar como profesor en unas aulas y como alumno en otras).
- Como leer el usuario actual desde cualquier componente (ej: Leo para autor del comentario):

```
const sesion = JSON.parse(localStorage.getItem('gruplab_usuario') || '{}');
```

8.2 Modelos compartidos entre Abel (Tablero) y Leonardo (Tareas)

1. frontend/src/app/features/tasks/models/board-column.ts (Abel):

```
export interface BoardColumn {
  id: number;
  group_id: number;
  name: string;
  position: number;
  color: string;
}
```

2. frontend/src/app/features/tasks/models/task.ts (Leonardo y Abel):

```
export interface Task {
  id: number;
  group_id: number;
  column_id: number;
  column_name?: string;
  assigned_to: number | null;
  assigned_name?: string | null;
  title: string;
  description: string | null;
  created_at: string;
}
```

8.3 Componentes que se incrustan en otros

Componente	Duenio	Selector	Inputs / Outputs	Se incrusta en
GroupList	Leandro	app-group-list	[classroomId]="a.id"	PanelAula (Luciano)
MemberList	Gonzalo	app-member-list	[classroomId]="a.id"	PanelAula (Luciano)
TaskCard	Abel	app-task-card	[tarea]="t" (moverColumna)="..." (seleccionar)="..."	Kanban (Abel)
TaskForm	Leonardo	app-task-form	[groupId]="groupId()" [columnId]="primeraColumnaId()" (creada)="cargarTablero()"	Kanban (Abel)
TaskDetail	Leonardo	app-task-detail	[tarea]="tareaSeleccionada()!" (cerrar)="..." (actualizada)="cargarTablero()"	Kanban (Abel)
Comments	Leonardo	app-comments	[taskId]="tarea().id"	TaskDetail (Leonardo)

---

9. Modulo 1 - Luciano - Aulas y Home (COMPLETADO)

Todo el modulo 1 ya se encuentra finalizado, documentado y compilando sin errores:
- Backend (classroom.*):
- GET /classrooms, POST /classrooms, GET /classrooms/code/:join_code, GET /classrooms/:id, DELETE /classrooms/:id.
- Al crear un aula, registra automaticamente al docente en Members con rol 'profesor' y devuelve { message, aulaId, join_code, memberId }.
- Al eliminar un aula, elimina primero los miembros asociados (eliminarMiembrosPorAula) y luego el aula.
- Frontend (features/classrooms/):

- Rutas con lazy loading: /classrooms (Home), /classrooms/nuevo (Profesor), /classrooms/unirse (Alumno), /classrooms/:id (PanelAula).
- Manejo de "Mis AuLabs" en Home: tarjetas que distinguen si sos Profesor o Alumno en cada aula.
- PanelAula: muestra cabecera, codigo del aula, usuario conectado, boton "Eliminar Aula" (exclusivo para rol profesor) y deja listas las dos columnas para GroupList (Leandro) y MemberList (Gonzalo).

---

10. Modulo 2 - Leandro - Grupos

- Backend: Ya integrado y funcionando en group.routes.js, group.controller.js, group.service.js, group.repository.js y queries.js.
  - Endpoints activos:
- GET /classrooms/:classroomId/groups -> [{ id, classroom_id, group_number, project_name }]
    - GET /groups/:id -> { id, classroom_id, group_number, project_name }
    - POST /classrooms/:classroomId/groups -> Body: { group_number, project_name }
    - PUT /groups/:id -> Body: { project_name }
    - DELETE /groups/:id
- Frontend pendiente (features/groups/):
  - models/group.ts: Interfaz Group (id, classroom_id, group_number, project_name).
  - services/group.ts (GroupService con inject(HttpClient)).
- group-list/ (app-group-list con classroomId = input.required<number>()): lista los grupos del aula, permite crear uno nuevo y tiene el boton "Ver tablero" con [routerLink]="['/groups', grupo.id, 'board']".

---

11. Modulo 3 - Gonzalo - Miembros

- Backend: Ya integrado y funcionando en member.routes.js, member.controller.js, member.service.js, member.repository.js y queries.js.
  - Endpoints activos:
- GET /classrooms/:classroomId/members -> [{ id, classroom_id, group_id, group_name, nombre, email, rol, created_at }]
    - GET /groups/:groupId/members
    - POST /classrooms/:classroomId/members -> Body: { nombre, email, rol }
    - PUT /members/:id/group -> Body: { group_id }
    - DELETE /members/:id
- Frontend pendiente (features/members/):
  - models/member.ts: Interfaz Member.
  - services/member.ts (MemberService con inject(HttpClient)).
- member-list/ (app-member-list con classroomId = input.required<number>()): muestra miembros del aula, diferencia al profesor con un badge, y permite asignar grupo a cada alumno con un <select>.

---

12. Modulo 4 - Abel - Tablero Kanban y Columnas Dinamicas (`BoardColumns`)

Responsabilidad: Construir la tabla BoardColumns, el Backend del Tablero y sus Columnas (board.*) (creacion automatica de las 4 columnas iniciales, posibilidad de agregar nuevas columnas, renombrarlas, eliminarlas y mover tareas entre columnas) y el

Frontend del Tablero Kanban (BoardService, Kanban y TaskCard).

12.1 Base de Datos (Abel)

Agregar la tabla BoardColumns en backend/src/database/schema.sql (ver seccion 7).

12.2 Backend de Abel (`board.*` + `queries.js`)

1. En backend/src/database/queries.js, agregar la constante base y las queries de Abel:

```
const queryBaseSelectColumnas = `SELECT
    id,
    group_id,
    name,
    position,
    color
FROM BoardColumns`;

const queryBaseSelectTareas = `SELECT
    t.id,
    t.group_id,
    t.column_id,
    c.name AS column_name,
    t.assigned_to,
    m.nombre AS assigned_name,
    t.title,
    t.description,
    t.created_at
FROM Tasks t
INNER JOIN BoardColumns c ON c.id = t.column_id
LEFT JOIN Members m ON m.id = t.assigned_to`;
```

Y dentro del module.exports:

```
// Queries de Abel (Columnas del Tablero y movimiento de tareas)
obtenerColumnasPorGrupo: `${queryBaseSelectColumnas}
    WHERE group_id = @group_id
    ORDER BY position ASC`,

obtenerColumnaPorId: `${queryBaseSelectColumnas}
    WHERE id = @id`,

crearColumna: `INSERT INTO BoardColumns (group_id, name, position, color)
    OUTPUT INSERTED.id
    VALUES (@group_id, @name, @position, @color)`,

actualizarColumna: `UPDATE BoardColumns
    SET name = @name, color = @color
    WHERE id = @id`,

eliminarColumna: `DELETE FROM BoardColumns WHERE id = @id`,

obtenerTareasPorGrupo: `${queryBaseSelectTareas}
    WHERE t.group_id = @group_id
    ORDER BY t.created_at ASC`,

moverTareaDeColumna: `UPDATE Tasks
    SET column_id = @column_id
    WHERE id = @id`,
```

2. Completar los 4 archivos de Backend de Abel:
   - backend/src/repositories/board.repository.js
   - backend/src/services/board.service.js
   - backend/src/controllers/board.controller.js
   - backend/src/routes/board.routes.js

Endpoints que expone Abel:

Metodo	URL	Body	Respuesta OK
GET	/groups/:groupId/board	-	{ columns: [...], tasks: [...] } (Si el grupo aun no tiene columnas, crea las 4 por defecto automaticamente)
POST	/groups/:groupId/columns	{ "name": "Testing", "color": "info" }	{ "message": "Columna creada exitosamente", "id": 5, "position": 5 }
PUT	/columns/:id	{ "name": "QA / Testing", "color": "info" }	{ "message": "Columna actualizada correctamente" }
DELETE	/columns/:id	-	{ "message": "Columna eliminada correctamente" }
PATCH	/tasks/:id/column	{ "column_id": 2 }	{ "message": "Tarea movida de columna correctamente" }

Logica de negocio en board.service.js:
- Inicializacion automatica de las 4 columnas base:
En obtenerTablero(groupId), consultar obtenerColumnasPorGrupo(groupId). Si devuelve 0 columnas, crear automaticamente las 4 columnas iniciales:
1. 'Por hacer' (position: 1, color: 'secondary')
2. 'En progreso' (position: 2, color: 'primary')
3. 'En revision' (position: 3, color: 'warning')
4. 'Terminado' (position: 4, color: 'success')
Luego devolver { columns, tasks }.
- Crear columna nueva (POST /groups/:groupId/columns):
Validar que name no este vacio (maximo 50 caracteres). Calcular position = columnasActuales.length + 1 y guardar en BoardColumns.
- Mover tarea de columna (PATCH /tasks/:id/column):
Verificar que la columna destino (column_id) exista con obtenerColumnaPorId(column_id) antes de actualizar la tarea.

3. Registrar rutas en backend/src/index.js:

```
const boardRoutes = require('./routes/board.routes');
// ...
app.use(boardRoutes);
```

12.3 Frontend de Abel (`features/tasks/`)

Archivos que construye Abel:
- frontend/src/app/features/tasks/models/board-column.ts (BoardColumn)
- frontend/src/app/features/tasks/services/board.ts (BoardService)
- frontend/src/app/features/tasks/kanban/ (kanban.ts, kanban.html, kanban.css)
- frontend/src/app/features/tasks/task-card/ (task-card.ts, task-card.html, task-card.css)
- En frontend/src/app/app.routes.ts: agregar la ruta con carga perezosa antes de '':

```
{
  path: 'groups/:groupId/board',
  loadComponent: () => import('./features/tasks/kanban/kanban').then(m => m.Kanban)
}
```

1. Servicio BoardService (features/tasks/services/board.ts):
- obtenerTablero(groupId: number): Observable<{ columns: BoardColumn[]; tasks: Task[] }>
- crearColumna(groupId: number, name: string, color?: string): Observable<any>
- actualizarColumna(columnId: number, name: string, color: string): Observable<any>
- eliminarColumna(columnId: number): Observable<any>
- moverTareaDeColumna(taskId: number, columnId: number): Observable<any>

2. Componente Kanban (selector app-kanban):
- Carga inicial (ngOnInit):

- Lee groupId con this.route.params.subscribe(...).
- Carga el grupo (GET /groups/:id) para mostrar *"Grupo #{{ g.group_number }} - {{ g.project_name }}"* y el boton "Volver al aula".
- Llama a boardService.obtenerTablero(groupId) y guarda columnas = signal<BoardColumn[]>([]) y tareas = signal<Task[]>([]).
- Renderizado dinamico de las columnas (@for (col of columnas(); track col.id)):
- Muestra las 4 columnas iniciales (y cualquier columna nueva que agregue el grupo).
- En cada columna filtra las tareas con obtenerTareasDeColumna(col.id) (respetando tambien el filtro por miembro responsable).
- Incluye boton "+ Nueva Columna" en la cabecera del tablero para crear una columna adicional al instante, y boton para eliminar/renombrar columnas.
- Integracion con los componentes de Leonardo:
- Boton "Nueva Tarea": abre <app-task-form [groupId]="groupId()" [columnId]="columnas()[0].id" (creada)="cargarTablero()" />.
- Clic en una tarjeta: abre <app-task-detail [tarea]="tareaSeleccionada()!" (cerrar)="tareaSeleccionada.set(null)" (actualizada)="cargarTablero()" />.

3. Componente TaskCard (selector app-task-card):
- Entradas: tarea = input.required<Task>(), puedeRetroceder = input<boolean>(true), puedeAvanzar = input<boolean>(true).
- Salidas: mover = output<'anterior' | 'siguiente'>(), seleccionar = output<Task>().
- Muestra titulo, badge del responsable (tarea().assigned_name || 'Sin asignar') y botones (<- y ->) para pasar la tarea a la columna anterior o siguiente en orden de position.

---

13. Modulo 5 - Leonardo - ABM de Tareas y Comentarios (`Tasks` y `Comments`)

Responsabilidad: Construir las tablas Tasks y Comments, el Backend de ABM de Tareas (task.*) y Comentarios (comment.*), los servicios TaskService y CommentService, y los componentes TaskForm, TaskDetail y Comments.

13.1 Base de Datos (Leonardo)

Agregar las tablas Tasks y Comments al final de backend/src/database/schema.sql (ver seccion 7).

13.2 Backend de Leonardo (`task.*`, `comment.*` y `queries.js`)

1. En backend/src/database/queries.js, agregar dentro del module.exports:

```
// Queries de Leonardo - ABM de Tareas
obtenerTareaPorId: `${queryBaseSelectTareas}
    WHERE t.id = @id`,

crearTarea: `INSERT INTO Tasks (group_id, column_id, assigned_to, title, description)
    OUTPUT INSERTED.id
    VALUES (@group_id, @column_id, @assigned_to, @title, @description)`,

actualizarTarea: `UPDATE Tasks
    SET title = @title, description = @description, assigned_to = @assigned_to
    WHERE id = @id`,

eliminarTarea: `DELETE FROM Tasks WHERE id = @id`,

// Queries de Leonardo - Comentarios
obtenerComentariosPorTarea: `SELECT id, task_id, author_name, content, created_at
    FROM Comments
    WHERE task_id = @task_id
    ORDER BY created_at ASC`,
```

```
    crearComentario: `INSERT INTO Comments (task_id, author_name, content)
        OUTPUT INSERTED.id
        VALUES (@task_id, @author_name, @content)`,

    obtenerComentarioPorId: `SELECT id, task_id, author_name, content, created_at
        FROM Comments
        WHERE id = @id`,

    eliminarComentario: `DELETE FROM Comments WHERE id = @id`,
```

2. Completar archivos de Tareas (task.repository.js, task.service.js, task.controller.js, task.routes.js):

Metodo	URL	Body	Respuesta OK
GET	/tasks/:id	-	{ id, group_id, column_id, column_name, assigned_to, assigned_name, title, description, created_at }
POST	/groups/:groupId/tasks	{ "column_id": 1, "title": "Armar DER", "description": "...", "assigned_to": 1 }	{ "message": "Tarea creada exitosamente", "id": 10 }
PUT	/tasks/:id	{ "title": "...", "description": "...", "assigned_to": 1 }	{ "message": "Tarea actualizada correctamente" }
DELETE	/tasks/:id	-	{ "message": "Tarea eliminada correctamente" }

Validaciones en task.service.js:
- title obligatorio (con .trim()), maximo 150 caracteres.
- column_id obligatorio al crear la tarea.
- En GET, PUT y DELETE, verificar primero que la tarea exista con obtenerTareaPorId(id); si no existe, lanzar throw new Error('Tarea no encontrada').

3. Completar archivos de Comentarios (comment.repository.js, comment.service.js, comment.controller.js, comment.routes.js):

Metodo	URL	Body	Respuesta OK
GET	/tasks/:taskId/comments	-	[{ id, task_id, author_name, content, created_at }]
POST	/tasks/:taskId/comments	{ "author_name": "Lautaro", "content": "Listo el DER" }	{ "message": "Comentario agregado exitosamente", "id": 1 }
DELETE	/comments/:id	{ "author_name": "Lautaro" }	{ "message": "Comentario eliminado correctamente" }

Validaciones en comment.service.js:
- author_name y content obligatorios (no vacios, maximo 1000 caracteres).
- Solo el autor puede eliminar su propio comentario (if (comentario.author_name !== author_name) throw new Error('Solo el autor puede eliminar este comentario')).

4. Registrar en backend/src/index.js:

```
const taskRoutes = require('./routes/task.routes');
const commentRoutes = require('./routes/comment.routes');
// ...
app.use(taskRoutes);
app.use(commentRoutes);
```

13.3 Frontend de Leonardo (`features/tasks/` y `features/comments/`)

1. Modelos:
- frontend/src/app/features/tasks/models/task.ts: Interfaz Task.

- frontend/src/app/features/comments/models/comment.ts: Interfaz Comment.

2. Servicios (inject(HttpClient) y Observable<T>):
- frontend/src/app/features/tasks/services/task.ts (TaskService):
- obtenerTareaPorId(id: number): Observable<Task>
- crearTarea(groupId: number, body: { column_id: number; title: string; description?: string; assigned_to?: number | null }): Observable<any>
- actualizarTarea(id: number, body: { title: string; description?: string; assigned_to?: number | null }): Observable<any>
- eliminarTarea(id: number): Observable<any>
- frontend/src/app/features/comments/services/comment.ts (CommentService):
- obtenerComentarios(taskId: number): Observable<Comment[]>
- agregarComentario(taskId: number, author_name: string, content: string): Observable<any>
- eliminarComentario(id: number, author_name: string): Observable<any>

3. Componente TaskForm (features/tasks/task-form/, selector app-task-form):
- Entradas: groupId = input.required<number>(), columnId = input.required<number>().
- Salidas: creada = output<void>(), cancelar = output<void>().
- Campos: title, description y <select> de miembro responsable del grupo (GET /groups/:groupId/members).
- Al guardar llama a taskService.crearTarea(...) y emite creada.emit().

4. Componente TaskDetail (features/tasks/task-detail/, selector app-task-detail):
- Entrada: tarea = input.required<Task>().
- Salidas: cerrar = output<void>(), actualizada = output<void>().
- Permite editar titulo, descripcion, responsable (taskService.actualizarTarea) o eliminar la tarea (taskService.eliminarTarea con confirm()).
- Abajo incluye el componente de comentarios: <app-comments [taskId]="tarea().id" />.

5. Componente Comments (features/comments/comments/, selector app-comments):
- Entrada: taskId = input.required<number>().
- Lee el nombre del usuario actual desde localStorage.getItem('gruplab_usuario').
- Muestra lista cronologica con fecha (DatePipe: {{ c.created_at | date:'dd/MM/yyyy HH:mm' }}).
- Muestra boton eliminar solo si c.author_name === usuarioActual().
- Textarea y boton "Comentar" que limpia el campo y recarga la lista.

---

14. Como trabajar sin esperar al companero (datos de prueba)

```
-- 1. Crear aula, grupo y miembro:
INSERT INTO Classrooms (name, duenio, join_code) VALUES ('Laboratorio Web', 'Lautaro',
'GL-1111');
INSERT INTO Groups (classroom_id, group_number, project_name) VALUES (1, 1, 'Sistema
Kanban');
INSERT INTO Members (classroom_id, group_id, nombre, rol) VALUES (1, 1, 'Lautaro',
'profesor');

-- 2. Crear las 4 columnas base del grupo 1 (Tabla BoardColumns de Abel):
INSERT INTO BoardColumns (group_id, name, position, color) VALUES
(1, 'Por hacer', 1, 'secondary'),
(1, 'En progreso', 2, 'primary'),
(1, 'En revision', 3, 'warning'),
(1, 'Terminado', 4, 'success');

-- 3. Crear tareas en distintas columnas (Tabla Tasks de Leonardo):
INSERT INTO Tasks (group_id, column_id, assigned_to, title, description) VALUES
(1, 4, 1, 'Armar tablas SQL', 'Crear BoardColumns, Tasks y Comments'),
(1, 3, 1, 'Endpoints de Tasks', 'Repository, Service y Controller'),
```

```
(1, 2, 1, 'Maquetar columnas dinamicas', 'Usar Bootstrap 5.3'),
(1, 1, NULL, 'Agregar filtros', 'Filtrar por miembro responsable');

-- 4. Comentario de prueba (Tabla Comments de Leonardo):
INSERT INTO Comments (task_id, author_name, content) VALUES (1, 'Lautaro', 'Tablas
verificadas sin error 1785');
```

---

15. Checklist de entrega por integrante

Comun a todos

- [ ] Codigo limpio siguiendo las convenciones de las secciones 5 y 6 (inject(), Signals, Observable<T>, Bootstrap 5.3).
- [ ] Todas las respuestas de error del backend en formato { error: error.message } con status 400.
- [ ] npm run build dentro de frontend ejecutando con 0 errores.
- [ ] Pull Request hacia develop sin conflictos.

Luciano (Modulo 1 - Aulas y Home)

- [x] Tabla Classrooms y Backend completo (classroom.*) con creacion de profesor en Members y borrado en cascada.
- [x] Frontend con Lazy Loading, RouterLink, inject(), Observable<T> e interfaces.
- [x] Manejo de sesion y "Mis AuLabs" multirrol en ClassroomService.
- [x] PanelAula con permisos de profesor y espacio para GroupList y MemberList.

Leandro (Modulo 2 - Grupos)

- [x] Tabla Groups y Backend (group.*) integrado y probado en peticionesTest.http.
- [ ] GroupService e interfaz Group en features/groups/.
- [ ] GroupList y creacion de grupo con navegacion a /groups/:groupId/board.

Gonzalo (Modulo 3 - Miembros)

- [x] Tabla Members y Backend (member.*) integrado y probado en peticionesTest.http.
- [ ] MemberService e interfaz Member en features/members/.
- [ ] MemberList con asignacion de grupo mediante <select> y baja de miembro.

Abel (Modulo 4 - Tablero Kanban y Columnas Dinamicas - Backend + Frontend)

- [ ] Tabla BoardColumns agregada a schema.sql.
- [ ] Queries y endpoints en board.repository.js, board.service.js, board.controller.js y board.routes.js (inicializacion automatica de las 4 columnas base, crear nueva columna, editar, eliminar y mover tarea de columna).
- [ ] BoardService en features/tasks/services/board.ts y ruta /groups/:groupId/board en app.routes.ts.
- [ ] Componentes Kanban (renderizado de las 4 columnas + boton "+ Nueva Columna" + filtros) y TaskCard (movimiento entre columnas).

Leonardo (Modulo 5 - ABM de Tareas y Comentarios - Backend + Frontend)

- [ ] Tablas Tasks y Comments agregadas a schema.sql.
- [ ] Queries y endpoints de ABM de tareas (task.*: GET /tasks/:id, POST /groups/:groupId/tasks, PUT /tasks/:id, DELETE /tasks/:id) y comentarios (comment.*: GET, POST, DELETE) probados en peticionesTest.http.
- [ ] TaskService y CommentService tipados con Observable<T>.
- [ ] Componentes TaskForm, TaskDetail y Comments (con autor desde localStorage y borrado solo por el autor).